

CampusEats – Service Contracts

Operations exposed between services, based on the supplied service design diagram. Each contract exposes only the operation, input, output, and caller-visible errors.

1. Order Service — Main Service

Operation	Input	Output	Errors
placeOrder	Student/account reference; selected food items and quantities; delivery address; payment choice.	Accepted order summary, total amount, payment status, delivery status, and current order status.	Invalid item; item unavailable; invalid address; payment declined; order rejected.
getOrder	Order reference and requesting student/account reference.	Order details and current order status.	Order not found; access denied.
cancelOrder	Order reference and requesting student/account reference.	Updated order status confirming cancellation.	Order not found; cancellation not allowed; order already completed.
getOrderStatus	Order reference.	Current order status and relevant payment/delivery state.	Order not found; status unavailable.

2. Catalogue Service — Main Service

Operation	Input	Output	Errors
checkItem	Food-item reference and requested quantity.	Item availability, current price, and restaurant information.	Item not found; item unavailable; catalogue unavailable.
getMenu	Restaurant reference.	Available menu items with names, descriptions, prices, and availability.	Restaurant not found; menu unavailable.
getVendor	Restaurant reference or food-item reference.	Restaurant/vendor information associated with the requested item.	Restaurant not found; item not found; vendor unavailable.
checkAvailability	Food-item reference.	Current availability status.	Item not found; availability unavailable.

3. Payment Service

Operation	Input	Output	Errors
makePayment	Order reference, amount, and selected payment method.	Payment result and payment status.	Payment declined; invalid payment request; payment processing failure.
paymentStatus	Order reference.	Current payment status and transaction result.	Payment not found; status unavailable.

4. Delivery Service

Operation	Input	Output	Errors
assignRider	Order reference, delivery address, and delivery request.	Rider assignment and current delivery status.	Order not eligible; no rider available; assignment failed.
trackDelivery	Order reference.	Current delivery status and latest delivery location.	Delivery not found; tracking unavailable.

5. Notification Service

Operation	Input	Output	Errors
sendNotification	Recipient/account reference, message content, and notification event.	Notification acceptance result and delivery status.	Invalid recipient; unsupported notification; delivery failed.
getNotificationStatus	Notification reference.	Current notification delivery status.	Notification not found; status unavailable.

6. User (Accounts) Service

Operation	Input	Output	Errors
getUser	Student/account reference.	User profile information needed by an authorized caller.	User not found; access denied.
getAddress	Student/account reference and requested delivery-address reference.	Delivery address details.	Address not found; access denied.

Contract boundary: The contracts expose only operations, inputs, outputs, and errors. They do not expose database tables, ID formats, SQL, storage structures, internal algorithms, payment-provider details, rider-selection logic, or notification infrastructure.

CampusEats – Full Specification of *placeOrder*

Central operation of the Order Service. This specification follows the supplied CampusEats service design: Order Service coordinates Catalogue, Payment, Delivery, Notification, and User/Accounts services.

1. Operation Contract

Field	Specification
Operation	placeOrder
Owner	Order Service
Purpose	Accept a student's requested food order, validate the requested items, arrange payment, create the order, request rider assignment, and trigger an order notification.
Inputs	Student/account reference; selected cart/order items and quantities; delivery address; selected payment method; optional order notes.
Output	Accepted order result containing order information, total amount, payment status, delivery status, and current order status.

2. Inputs in Detail

- **Student/account:** identifies the student placing the order.
- **Order items:** the food items requested and their quantities.
- **Delivery address:** where the order should be delivered.
- **Payment choice:** the payment method selected by the student.
- **Optional notes:** any additional order instruction supported by the service.

3. Outputs in Detail

- **Order result:** confirms that the order was accepted.
- **Order information:** the ordered items and calculated total.
- **Payment status:** indicates whether payment was successfully processed.
- **Delivery status:** indicates the current delivery/assignment state.
- **Order status:** indicates the current lifecycle state of the order.

4. Error Cases

Error	When it occurs
Invalid item	A requested food item cannot be accepted by the Catalogue Service.
Item unavailable	A requested item is not currently available.
Invalid address	The supplied delivery address is invalid or cannot be used for delivery.
Payment declined	The Payment Service does not approve the requested payment.
Payment processing failure	The Payment Service cannot complete the payment operation.
No rider available	Delivery cannot assign a rider for the accepted order.
Order rejected	A required business condition prevents the order from being accepted.

5. Internal Processing That Must Be Hidden

- **Catalogue lookup and validation:** Order Service may call *checkItem()* to verify item availability and obtain the information needed to accept the requested items. The Catalogue Service's internal storage and validation logic remain private.
- **Order construction:** Order Service validates the request, calculates the order total, and creates/updates its internal cart and order state. Callers see the result, not the internal representation.
- **Payment coordination:** Order Service may call *makePayment()* and receive *paymentStatus()*. Payment-provider details, transaction processing, and payment internals are hidden.
- **Delivery coordination:** After an accepted order, Order Service may call *assignRider()*. Rider selection, delivery assignment logic, and location handling remain inside Delivery Service.
- **Notification coordination:** Order Service may call *sendNotification()* to notify the student about the order. Notification channels and

delivery mechanisms remain private.

- **Failure handling:** Internal exceptions, retries, private service messages, and partial implementation state are not exposed. The caller receives only the defined output or a defined error.

6. Information Explicitly Hidden from Callers

The caller must not depend on database tables, SQL, storage schemas, internal object structures, ID-generation formats, payment-provider implementation, rider-selection algorithms, notification infrastructure, internal retry logic, or the exact sequence of private service-to-service calls. These are implementation details behind the *placeOrder* contract and may change without changing the public operation.

7. Contract Summary

placeOrder(input) → accepted order result | defined error. The caller knows what information to provide and what result or error to expect, while the Order Service remains free to change how it performs the work internally.